Foresight Fellowship 2027 - Christian Bucher Evidence Portfolio

Executive review route and evidence summary

Executive thesis

I built practical agent infrastructure around models: remote task ingress, Dual-Brain interpretation/evaluation, local daemon orchestration, model routing, memory governance, voice control and evidence-bounded research direction.

- Secure multiagent systems from a builder perspective.
- Evidence-first portfolio with explicit claim boundaries.
- Future project path: uncertainty-aware AI for antimicrobial resistance.

Reviewer route

- 5 minutes: remote/Dual-Brain, Miguel Core, cascade router.
- 20 minutes: full AI systems dossier and technical appendix.
- Deep review: ECSA future project and individual PDF dossiers.

Claim discipline

The package separates built prototypes, tested kernels, designed projects and speculative work. It avoids public local paths, credentials, private channel details and unverified medical/legal/physics claims.

Remote WhatsApp-to-Agent Command Loop

A phone should be able to issue work to a home computer without turning the system into an unsafe direct shell. The design had to translate casual WhatsApp messages or audio into structured work orders, run them through a tool-enabled agent side, judge completion, and only then report back.

- This is the clearest Secure AI signal: remote natural-language instructions became structured work orders, then passed through tool execution and evaluation before a user-facing reply.
- Prototype/evolved system. Some EXOVIUM pieces were later superseded by Miguel Core. Private channel identifiers and credentials are excluded.

What I had in mind

What I had in mind was simple but technically demanding: I wanted to be away from my laptop and still delegate meaningful work to it without turning my phone into a blind remote shell. The interesting part was the trust boundary. A message had to become a work order, the work order had to be executed by the right side of the system, and the result had to survive evaluation before coming back to me.

What I built

- Implemented across OpenClaw watcher, MiguelAgent, EXOVIUM core, WA task manager, cognition routing, and architecture notes. Source notes document immediate ACK behavior, quality-loop thresholding, model routing, audio transcription, and two successful end-to-end tests.

Mechanism detail

- The valuable idea is not WhatsApp itself. WhatsApp is only the low-friction ingress. The hard part is the translation layer: casual phone input becomes a structured work order with goal, complexity, model hint and success criteria.
- The Dual-Brain pattern separates the role that understands/evaluates from the role that executes with tools. That prevents a phone message from becoming an unchecked direct command.
- The response loop is deliberately delayed until the work is judged complete enough. This is closer to a supervisor/executor pattern than ordinary chatbot messaging.

Miguel Core Unified Daemon

Early systems had separate daemons, command scripts, and services. Miguel Core consolidated the control surface into a local Python daemon that could coordinate chat, streaming, voice, memory, model tiers, orchestration, state, and tool endpoints.

- This shows system-level building rather than model use: interfaces, state, memory, tools, routing, health, orchestration, and operating boundaries were treated as one control plane.
- Local experimental control plane, not a hardened commercial security product.

What I had in mind

Miguel Core came from a very practical frustration: I had too many strong pieces living as separate scripts and services. I wanted one local control layer that could make the system observable, routable and easier to reason about.

What I built

- Central file `miguel_core.py` documents the consolidation goal and exposes the route surface. `orchestra.py` implements async worker delegation to Opus, Codex, GPT-style workers through a subprocess layer with timeouts, cancellation, status, and stats.

Mechanism detail

- Miguel Core is best explained as a local agent operating layer. It consolidates chat, streaming, voice, state, memory, prediction, model routing, orchestration and tool endpoints into one observable control plane.
- The route surface matters because it shows the work moved from isolated scripts toward a system with health, state, conversation persistence and worker delegation.
- The surrounding startup and handover documents are part of the evidence: they define boot checks, operating modes, quality anchors, autonomy rings and state probes.

Multi-Model Cascade Router

Agent systems fail when all tasks are pushed through one model. The router treats models as resources with strengths, costs, quotas, cooldowns, and failure modes.

- This shows model governance in practice: models are resources with availability, cost, quality, quota, privacy, and failure characteristics.
- Cost-reduction claims should be treated as design goals unless fresh logs verify exact percentages.

What I had in mind

I did not want to treat model choice as taste. I wanted routing to be part of the architecture: cheap when possible, strong when necessary, local when privacy matters, and fallback-aware when a provider fails.

What I built

- `ai_router.py`, `cascade.py`, and `litellm_config.yaml` define model registries, routing strategies, tiering, quota tracking, fallback, race/cancel behavior, and budget-aware escalation.

Mechanism detail

- The router treats models as infrastructure resources with strengths, costs, quotas, cooldowns, privacy profiles and failure modes.
- Task strategy determines whether to prefer speed, quality, free models, local execution, code capability or paid escalation.
- The core safety value is graceful degradation: when one model or tier fails, the system has a policy for trying another path rather than silently returning weak work.